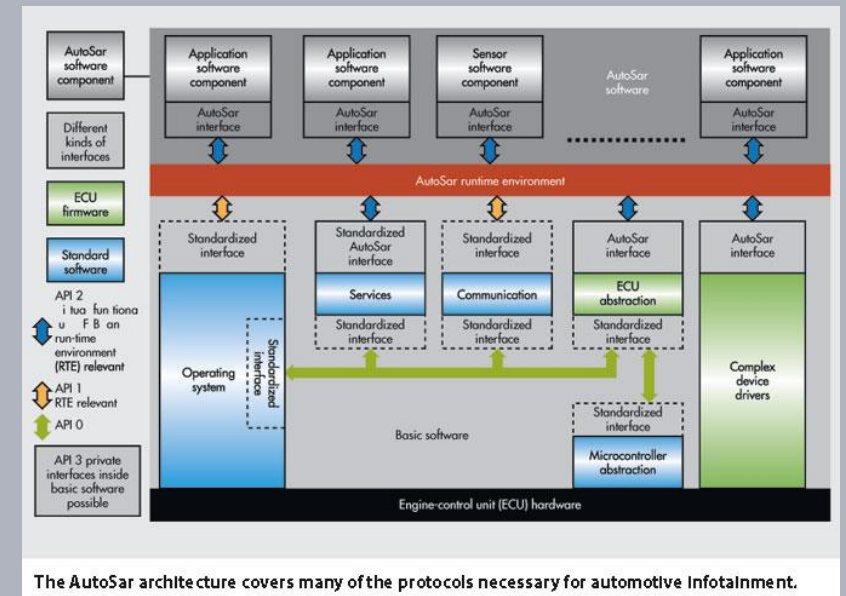


Advanced Microcontroller Architectures and Embedded OS

OSEK, Erika on PSOC

Prof. Dr.-Ing. Peter Fromm



The AutoSar architecture covers many of the protocols necessary for automotive Infotainment.

Content

- OSEK History
- Development Structure
- Demo Project - Blinker
- OSEK Tasks
- OSEK Counters and Alarms

OSEK History

OSEK (Offene Systeme und deren Schnittstellen für die Elektronik in Kraftfahrzeugen; English: "Open Systems and their Interfaces for the Electronics in Motor Vehicles") is a standards body founded in 1993 by a German automotive company consortium (BMW, Robert Bosch GmbH, DaimlerChrysler, Opel, Siemens, and Volkswagen Group) and the University of Karlsruhe. In 1994, the French cars manufacturers Renault and PSA Peugeot Citroën joined the consortium.

Major components

- OSEK-OS (Operating System kernel)
- OSEK-OIL (OSEK Implementation Language)
- OSEK-ORTI (OSEK Runtime Library)
- OSEK-COM (OSEK Communication)
- OSEK-NM (OSEK Network Management)
- OSEK-TIME (OSEK Realtime Extensions)

OSEK and AUTOSAR

AUTOSAR (AUTomotive Open System ARchitecture) is an open and standardized automotive software architecture, jointly developed by automobile manufacturers, suppliers and tool developers. It is a partnership founded 2003 of automotive OEMs, suppliers and tool vendors whose objective is to create and establish open standards for automotive E/E (Electrics/Electronics) architectures that will provide a basic infrastructure to assist with developing vehicular software, user interfaces and management for all application domains.

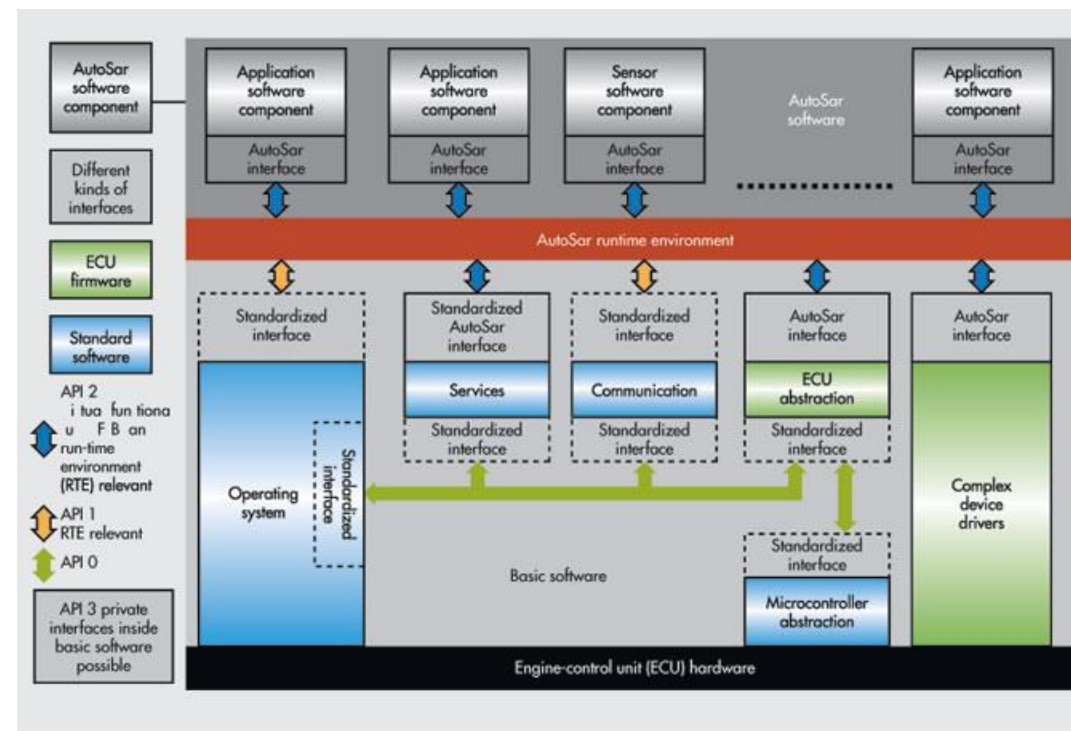
OSEK and Autosar

AUTOSAR has been devised to:

- pave the way for innovative electronic systems that further improve performance, safety and environmental friendliness
- be a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation"
- be a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality
- facilitate the exchange and update of software and hardware over the service life of the vehicle

OSEK and Autosar

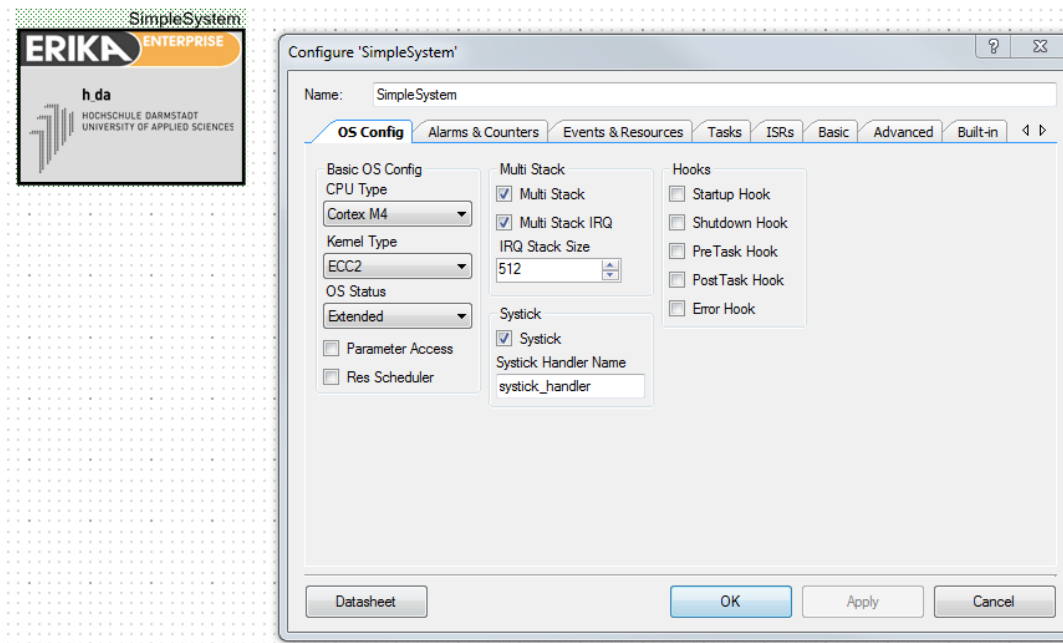
AUTOSAR follows the layered architecture concept. The total software solution is divided into 3 layers, known as Application software, Runtime Environment (RTE) and Basic Software (BSW). All the functionalities are kept under the application software module, the Run Time Environment acts as a virtual function bus for an Electronic control unit and messages are transferred using the basic software.



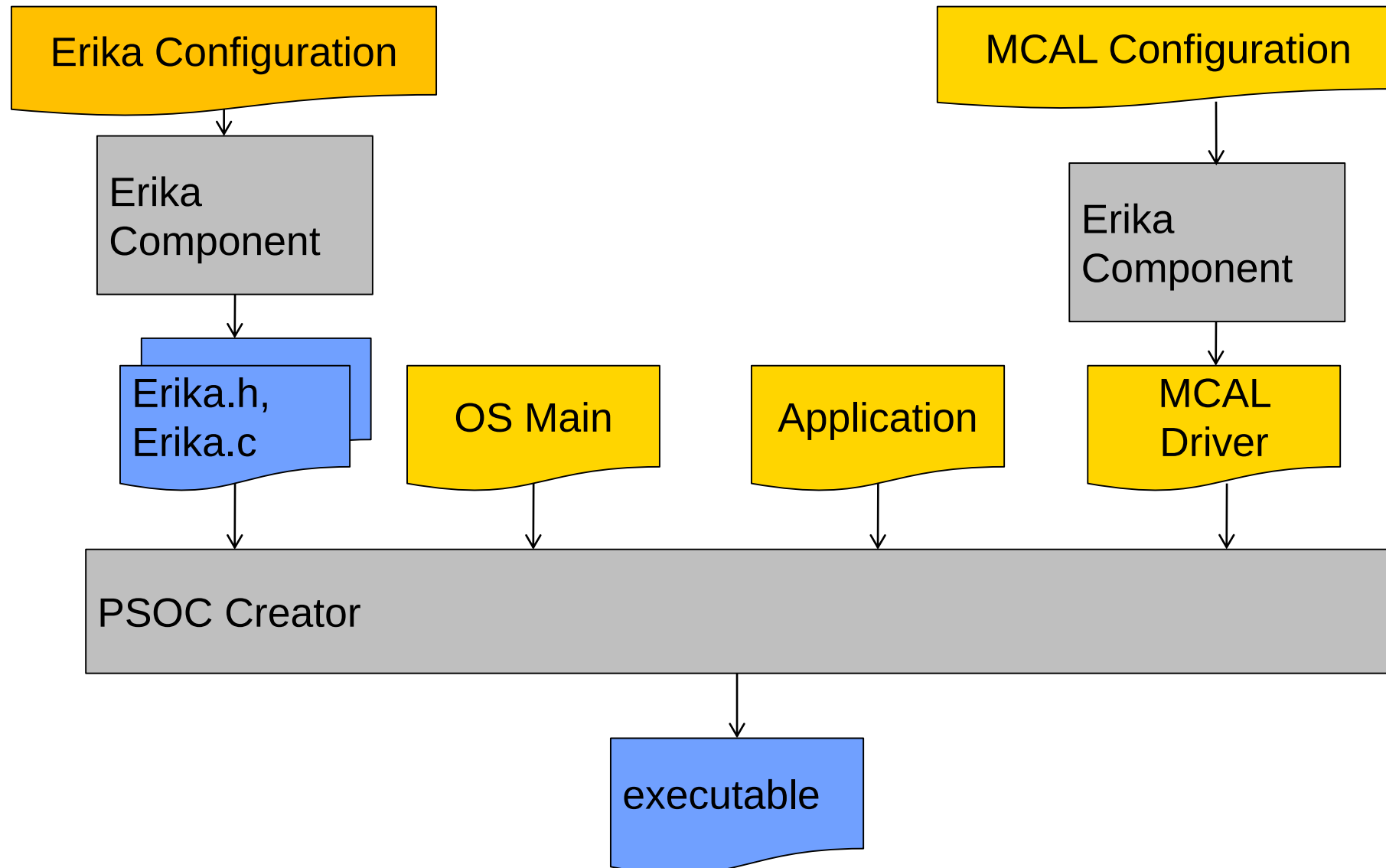
The AutoSar architecture covers many of the protocols necessary for automotive Infotainment.

Erika Enterprise is an open-source Free RTOS that has been certified OSEK/VDX compliant!

Own port for PSOC available. Instead of the RTDruid Configuration, an own PSOC Plugin has been developed. Credits to Carlos Meier!



Erika / Autosar Workflow



Example: Turn Indicator

Task: Development of the control unit for a turn indicator

Requirements:

- If the left switch is pressed, the left indicator will blink with a frequency of 1Hz
- If the right switch is pressed, the right indicator will blink with a frequency of 1Hz
- If the emergency switch is activated, both indicators will blink
- Tipping the left/right switch will let the blinker blink for 5s



Changing development paradigm

Iterative development in sequential code

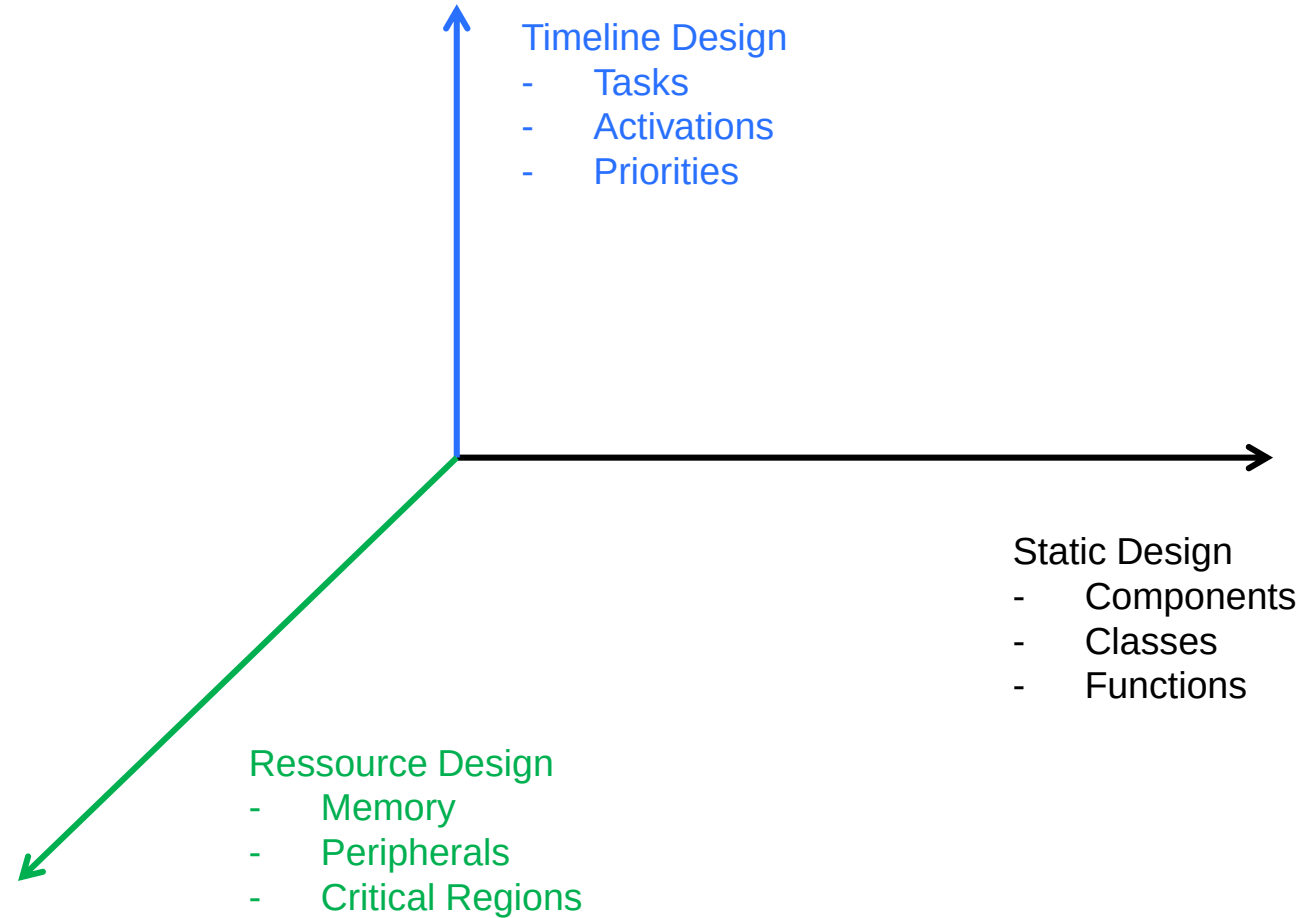
- independant functions can be implemented one after the other, but only, if EITHER the one OR the other is executed
- dependencies between different functions are usually implemented using a classic client/server call pattern (i.e. normal function call) – tight coupling!

Iterative development in multithreading code

- The syntax is more complex and problems are harder to debug but...
- also parallel executed functions can be implemented one after the other
- events allow loose coupling

In our example: „overtake“ logic can be added later...

Design Dimensions

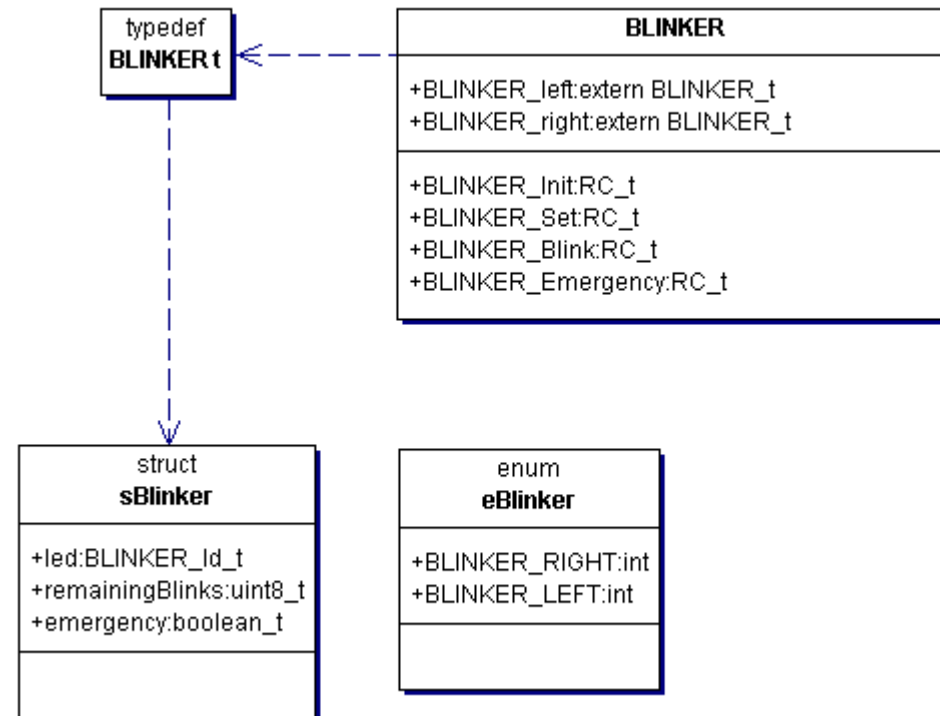


Analysing Requirements

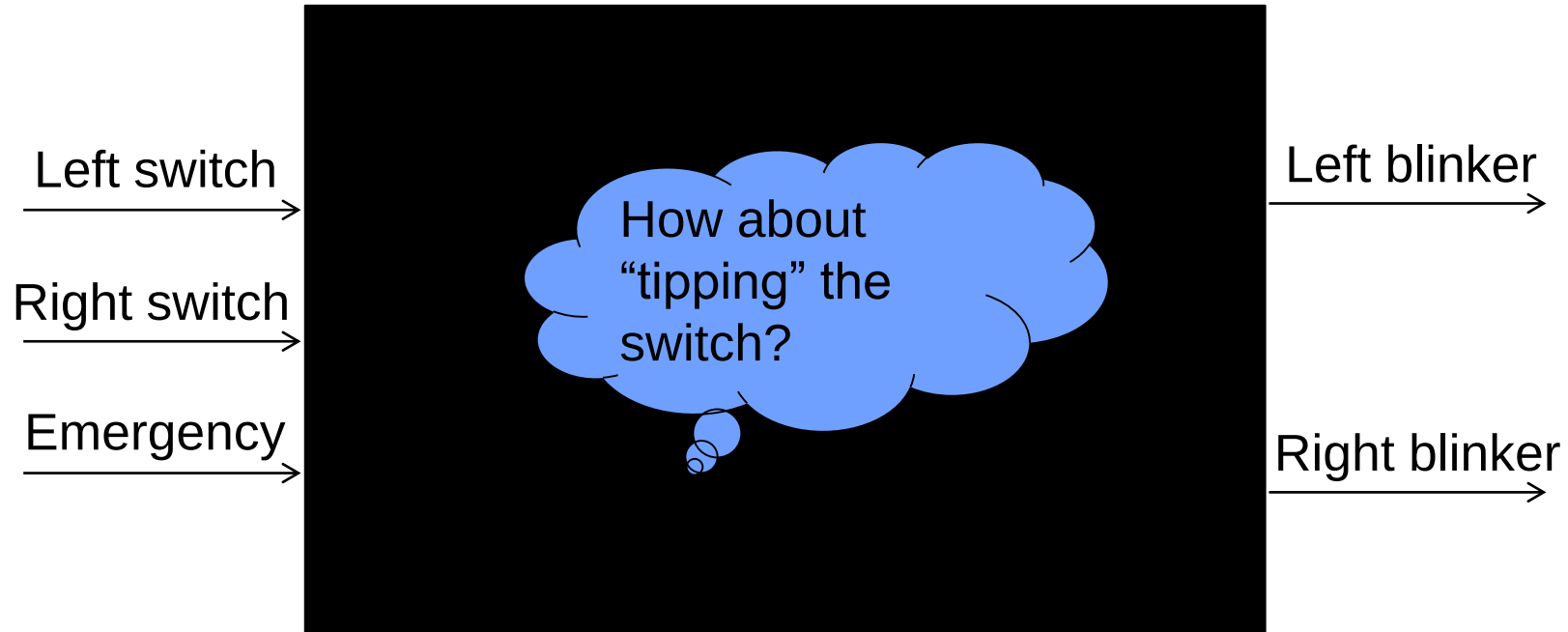
Object Oriented Analysis and Design

How shall we describe the blinker and design the blinker API?

- We have two blinker objects
- Which can be on or off
- Besides turning them explicitly on or off, they also have a kind of a count down logic for the overtake button
- In the case of an emergency button press they should remain on

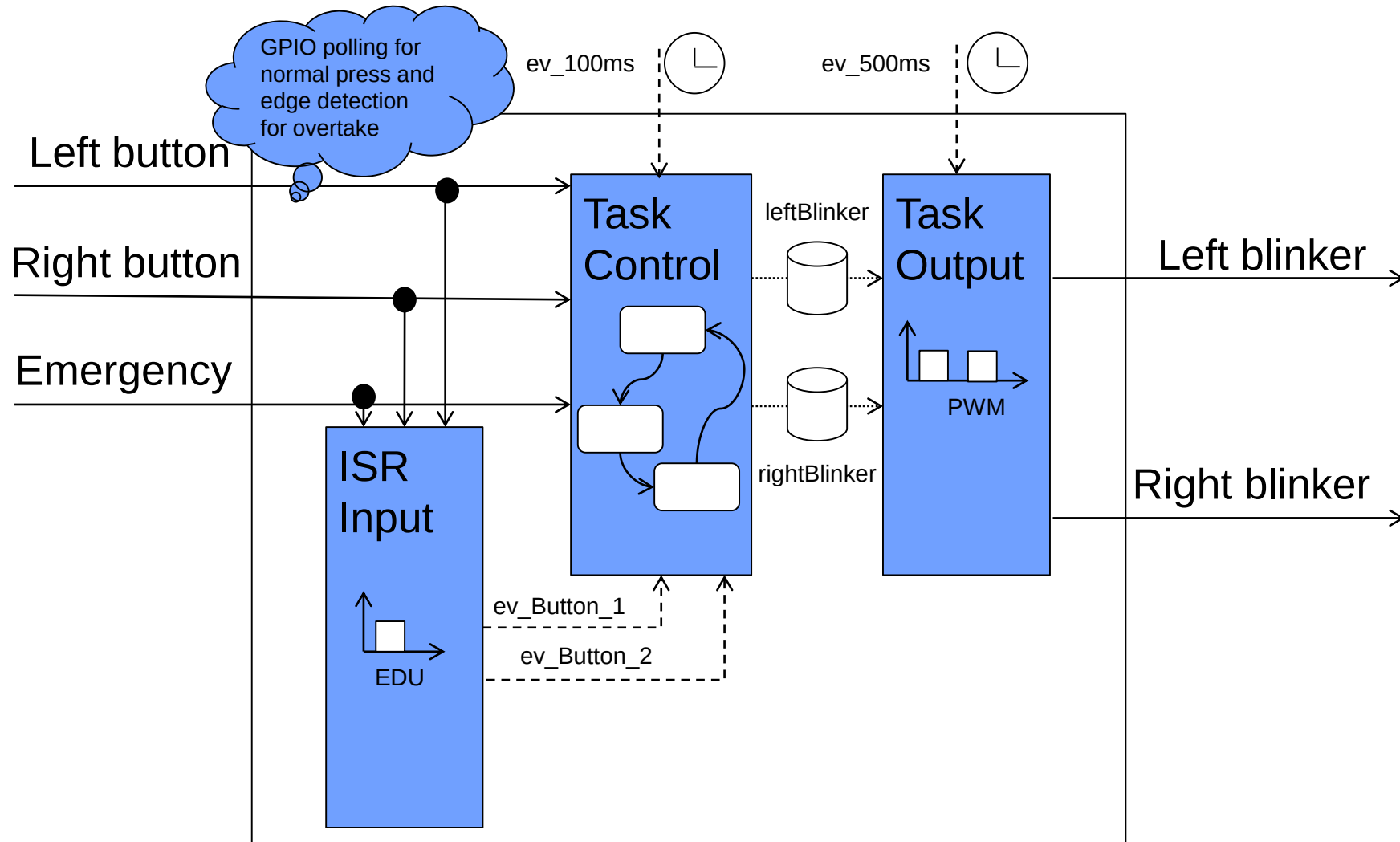


Analysing Requirements Signals on Hardware Level



Analysing Requirements

Tasks, Triggers and Signalflow



Iteration 1 – Blinking LED's

Some OSEK mechanisms

- **Tasks**, which can either be activated by
 - Software activation
 - or cyclic alarms
 - or events
- **Alarms** represent software timers which
 - can fire cyclic tasks
 - or fire events
 - or are fired only once

Which activities are executed in independant time domains?
What „timing requirements“ do we have?
Which tasks do we need? How are they activated?

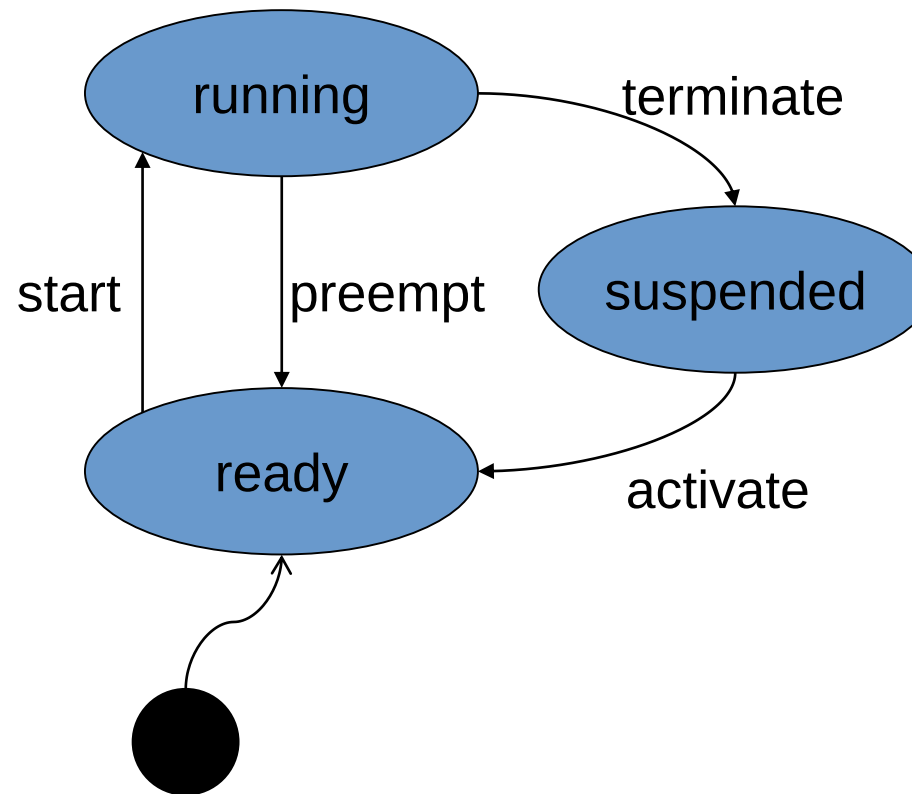
General Task Structure

- InitTask is responsible for peripheral initialisation and task activation
- Background tasks serves for debugging - while(1)
- Basic Tasks for simple cyclic jobs
- Extended Tasks for event processing

Task states – OSEK

Basic Task

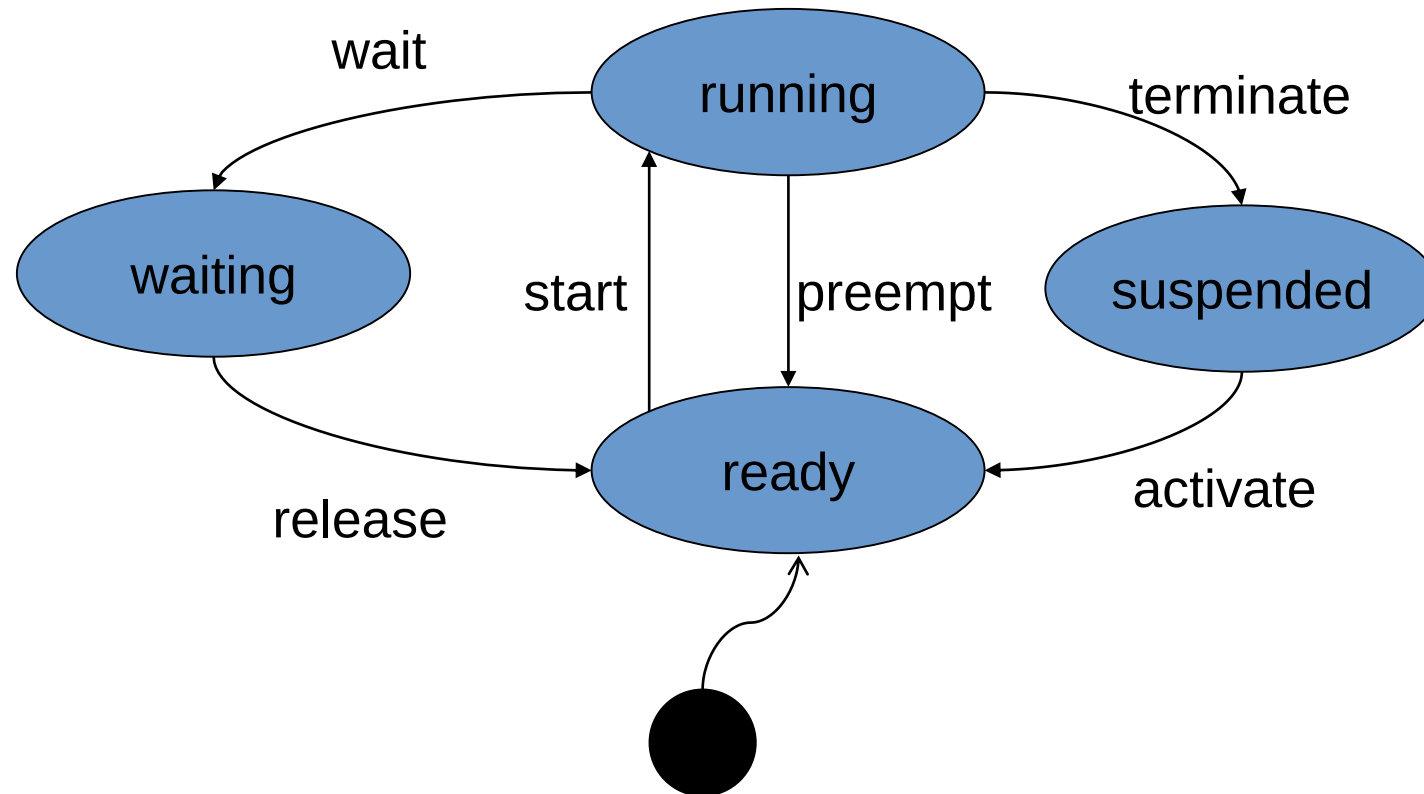
Basic tasks – not supporting events:



Task states – OSEK

Extended Task

Extended tasks – supporting events:



Typical Task Structure

Basic Task

- Triggered by an Alarm or manual activation
- While (1) loop only for background task and lowest priority
- OSEK does not provide delay()

```
TASK(tsk_basic)
{
    //Do Something

    TerminateTask();
}
```

Extended Task

- Runs in endless loop
- Wake Up through events
- Often implemented as state machine

```
TASK(tsk_extended)
{
    while(1) {
        WaitEvent();
        GetEvent();
        ClearEvent();

        //Statemachine
    }
    TerminateTask();
}
```

Task states – OSEK

Example Extended Task

```
TASK(tsk_Basic)
{
    EventMaskType ev = 0;

    DoSomeApplicationInitialization();

    while(1)
    {
        //This is a scheduling point. Task will sleep until event is fired
        WaitEvent(ev_1 | ev_2 | ev_...);
        //Read out the event mask (to know, which event was fired)
        GetEvent(tsk_Control, &ev);
        //Clear the bitmask to allow the next event to be received
        ClearEvent(ev);

        //Bitwise AND
        if (ev & ev_1) DoSomething();
    }

    TerminateTask();
}
```

OSEK Tasks

Extended tasks
need private
stack

Configure 'OS'

Name: OS

OS Config | Alarms & Counters | Events & Resources | **Tasks** | ISRs | Basic | Advanced | Built-in

tsk_Init | tsk_Control | tsk_Background | tsk_Output

Task Name: tsk_Init

Task Priority: Priority 1

Task Schedule: Non

☒ Autostart

☐ Private Stack

Stack Size: 0

Task Activation: 1

Event List:

- ☐ ev_BlinkerOvertakeI
- ☐ ev_BlinkerOvertakeF
- ☐ ev_100ms

Resource List:

Add Remove

Datasheet OK Apply Cancel

Events which
can activate the
task

OSEK Tasks

Priority

- higher value = higher priority
- higher priority tasks preempt lower priority tasks

Activation

- Number of possible activations at the same time

Schedule

- Full: Preemptive
- None: Non-preemptive

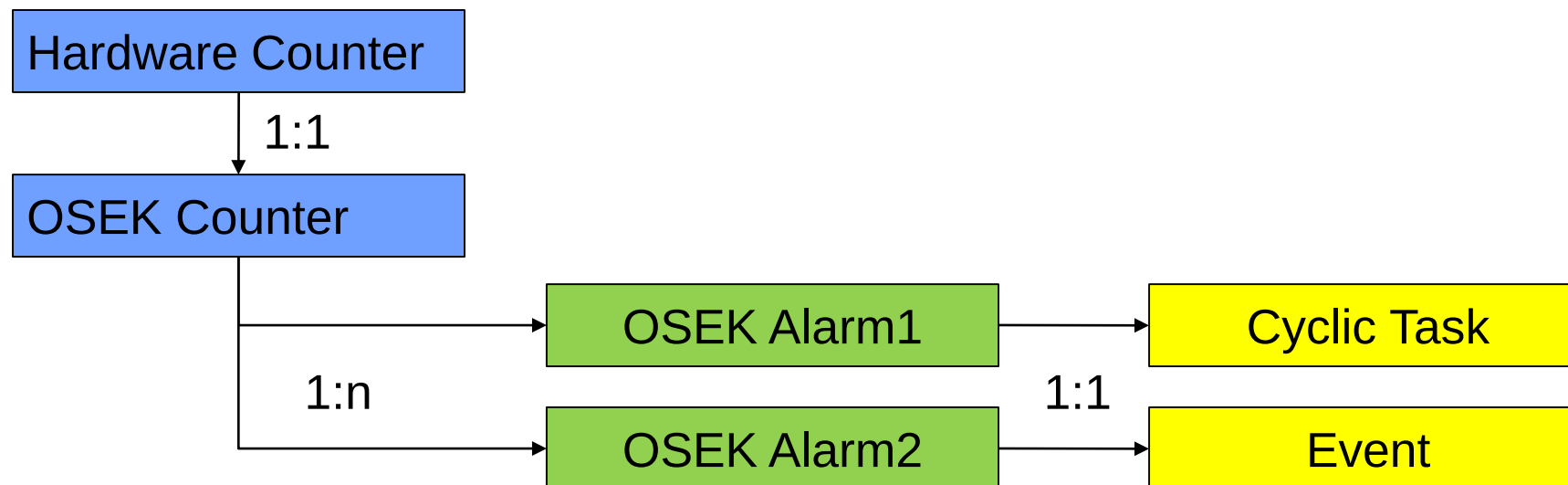
Stacksize

- Reserved stacksize for task

Counters and Alarms

Background

- Counters are based on hardware counters
- Alarms are triggered by Counters and trigger events or tasks themselves
- Several alarms can be based on one counter, firing their task at $N \times \text{counter time}$
- An alarm is connected to one dedicated task or event



Counter Configuration

Configure 'OS'

Name: OS

OS Config **Alarms & Counters** Events & Resources Tasks ISRs Basic Advanced Built-in

cnt_systick

Counter Name
cnt_systick

Counter Tick
0

Min Cycle
1

Max Value
10000

Add Remove

alm_100ms alm_500ms

Alarm Name	Callback Name
alm_100ms	

Counter	Task
cnt_systick	tsk_Control

Alarm Action
Set Event

Event
ev_100ms

Add Remove

Datasheet OK

```
20
21 //ISR which will increment the systick counter every ms
22 ISR(systick_handler)
23 {
24     CounterTick(cnt_systick);
25 }
26
27 int main()
28 {
29     CyGlobalIntEnable; /* Enable global interrupts. */
30
31     //Set systick period to 1 ms. Enable the INT and start it.
32     EE_systick_set_period(MILLISECONDS_TO_TICKS(1, BCLK_BUS_CLK_HZ));
33     EE_systick_enable_int();
34     EE_systick_start();
35
36     // Start Operating System
37     for(;;)
38         StartOS(OSDEFAULTAPPMODE);
39 }
```


Alarm API

//Declaration

```
AlarmBaseType albase_blink;
```

// Get the alarm base from the OIL configuration (old API – typically not required anymore)

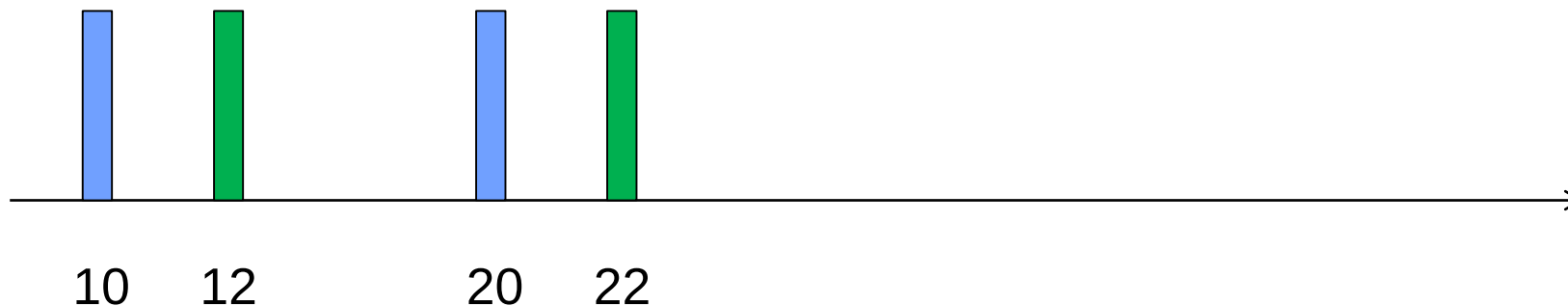
```
GetAlarmBase(alm_blink, &albase_blink);
```

// Start the alarm

```
SetRelAlarm(alm_blink,  
            offset,      //offset  
            cyclic );    //subsequent periods
```

//Scheduletable

```
SetRelAlarm(blue, 10,10);  
SetRelAlarm(green, 12,10);
```



Wrap-Up

- OSEK History
- Development Process
- TASK structure
- Counters and Alarms